

Tema 2.3

1. El dilema del bucle asíncrono con var frente a let

```
for (var i = 0; i < 3; i++) {  
  setTimeout(function() {  
    console.log("Valor con var:", i);  
  }, 100);  
}  
  
for (let j = 0; j < 3; j++) {  
  setTimeout(function() {  
    console.log("Valor con let:", j);  
  }, 100);  
}
```

Salida tras los 100 ms:

```
Valor con var  
Valor con var  
Valor con var  
Valor con let  
Valor con let  
Valor con let
```

Con var sale el 3 tres veces porque var tiene ámbito de función o global y el bucle crea una sola variable i compartida por las tres funciones. Los setTimeout no se ejecutan hasta que termina el bucle, y para entonces i ya vale 3: las tres llamadas leen el mismo valor.

Con let sale 0, 1 y 2 porque let tiene ámbito de bloque y cada iteración del for crea su propia copia de j, una captura distinta por vuelta. Cada callback ve el valor de su iteración y el bucle no se lo pisa.

2. Identificación de la Zona Muerta Temporal (TDZ)

```
let usuario = "Carlos";  
function configurarPerfil() {  
  console.log("El usuario actual es:", usuario);  
  let usuario = "Marta";  
}  
configurarPerfil();
```

Salida:

```
Uncaught ReferenceError: Cannot access 'usuario' before initialization
```

No se imprime nada. El `let usuario` de dentro de la función ensombrece al global desde el primer byte de la función, no desde la línea de su declaración. Al llegar al `console.log`, la variable interna ya existe, pero está en su Zona Muerta Temporal (TDZ): el tramo entre la entrada al ámbito y la declaración física. El motor prohíbe leerla en ese estado y lanza `ReferenceError`.

Para que imprimiera habría que usar otro nombre (`let usuarioInterno = "Marta"`) o leer el usuario global con un identificador distinto.

3. Simulador visual de ámbitos

Modificación 1: simulación de sombreado (shadowing).

Constante global, declarada por encima del `addEventListener`:

```
const TASA = .;
```

Dentro del manejador del botón, antes de finalizar la auditoría:

```
function calcularTarifa(precio) {  
  const TASA = .;           // sombrea a la global solo dentro de es  
  ta función  
  return precio * TASA;  
}  
  
registrar("TASA global: " + TASA);           // 0.21 (la origin  
al no cambia)  
registrar("TASA local (dentro de la función): 0.10");  
registrar("Tarifa de 200 €: " + calcularTarifa()); // 20
```

Comprobación:

```
TASA global .  
TASA local (dentro de la funcin) .  
Tarifa de
```

La constante interna no altera la externa: son dos posiciones de memoria distintas y, al salir de la función, la local desaparece y la global sigue valiendo 0.21.

Modificación 2: activación de "use strict".

Primera línea del fichero `laboratorio_ambitos.js`:

```
"use strict";
```

Y dentro del manejador del botón, intento de fuga capturado:

```
try {  
  saldoSinDeclarar = ; // ni var, ni let, ni const  
  registrar("Se creó una variable global oculta.", "resaltado-ok");  
} catch (e) {  
  registrar(e.constructor.name + ": " + e.message, "resaltado-error");  
}
```

Salida real en la pantalla del laboratorio:

ReferenceError saldoSinDeclarar is not defined

Sin "use strict" esa línea no daría ningún error: crearía una variable global oculta en window (comprobado, `window.saldoSinDeclarar === 500`). Con la directiva activa, el motor lanza ReferenceError en cuanto se intenta la asignación, que es lo que captura el try/catch y pinta .resaltado-error.

Resumen de comprobaciones

Ejercicio	Resultado
1, bucle con var	Valor con var: 3 tres veces
1, bucle con let	0, 1, 2
2, TDZ	ReferenceError: Cannot access 'usuario' before initialization
3, sombreado de TASA	global 0.21, local 0.1, tarifa de 200 € da 20
3, "use strict"	ReferenceError: saldoSinDeclarar is not defined; sin la directiva crearía una global con 500